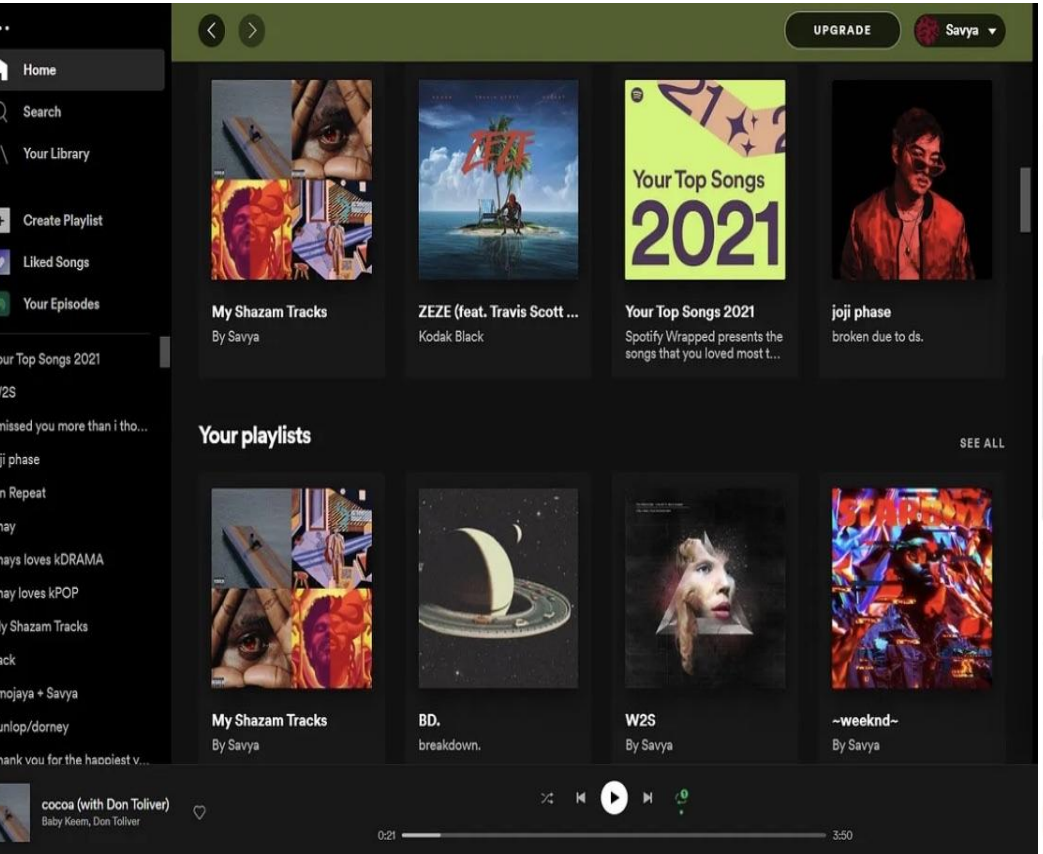
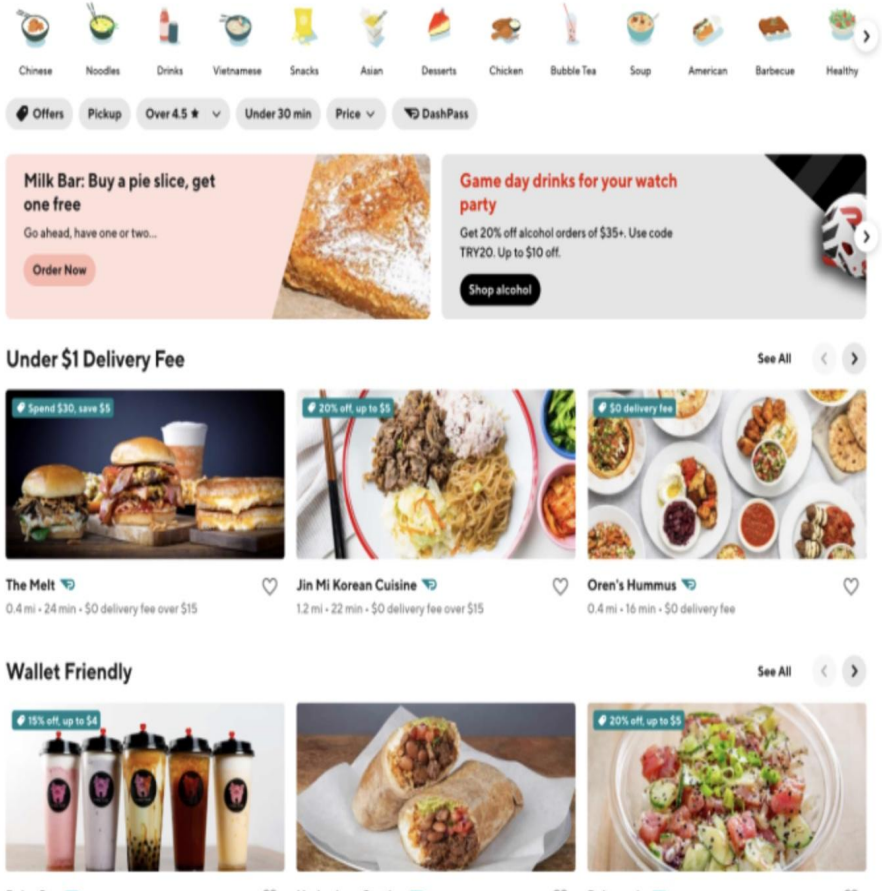
Measuring and Improving Native Execution in Apache DataFusion Comet

“DATA IS THE NEW OIL IN 21st CENTURY!”





- A database is an organized collection of structured information, or data, typically stored electronically in a computer system.
- A database is typically deployed and accessed via cloud environment.
- SQL is a programming language used to interact with the database.

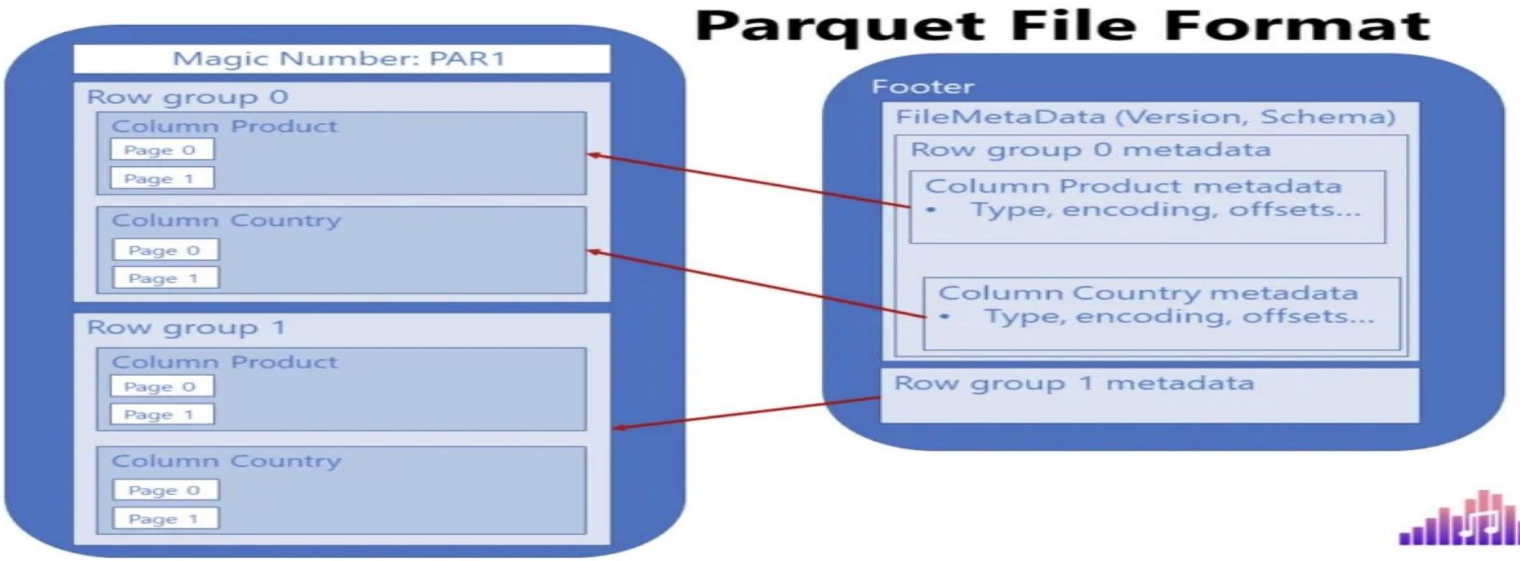
TABLE-1

Product ID	Product Name	Color ID
0001	Mattress	Color 1
0002	Pillow	Color 2

Color ID	Color Name
Color 1	Blue
Color 2	Red

TABLE-2

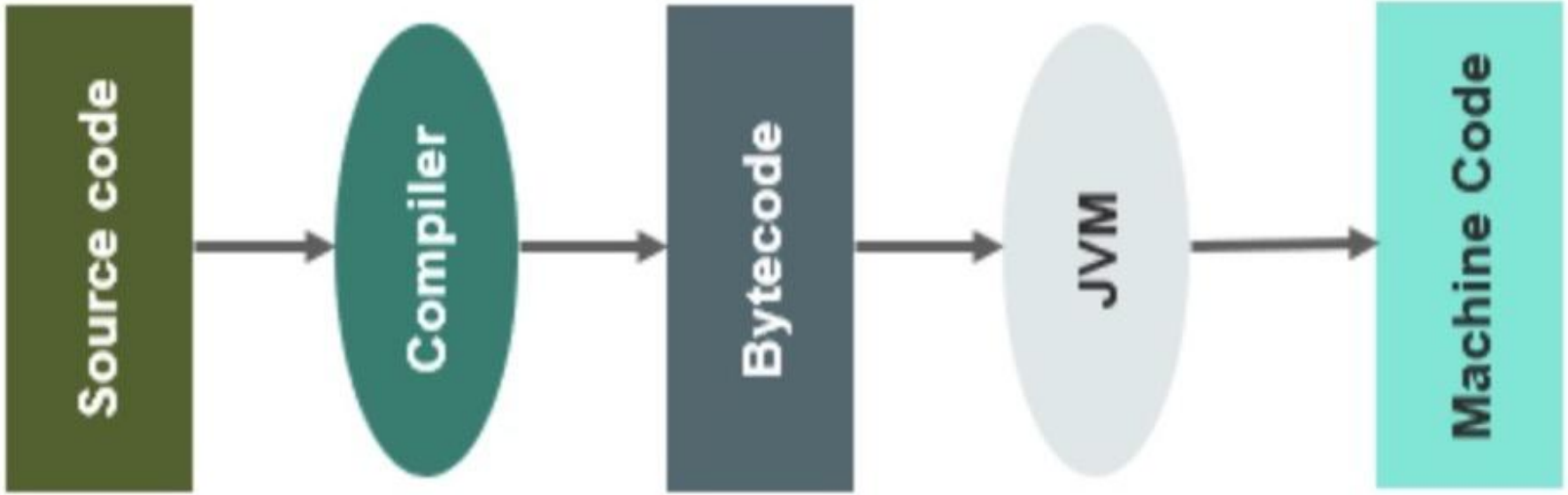
- Traditional Row based storage is very slow and memory inefficient.
- Parquet files store chunks of data in row-groups as well as metadata.
- The row-group format offers for more efficient memory usage through dictionary encoding.



Apache Spark

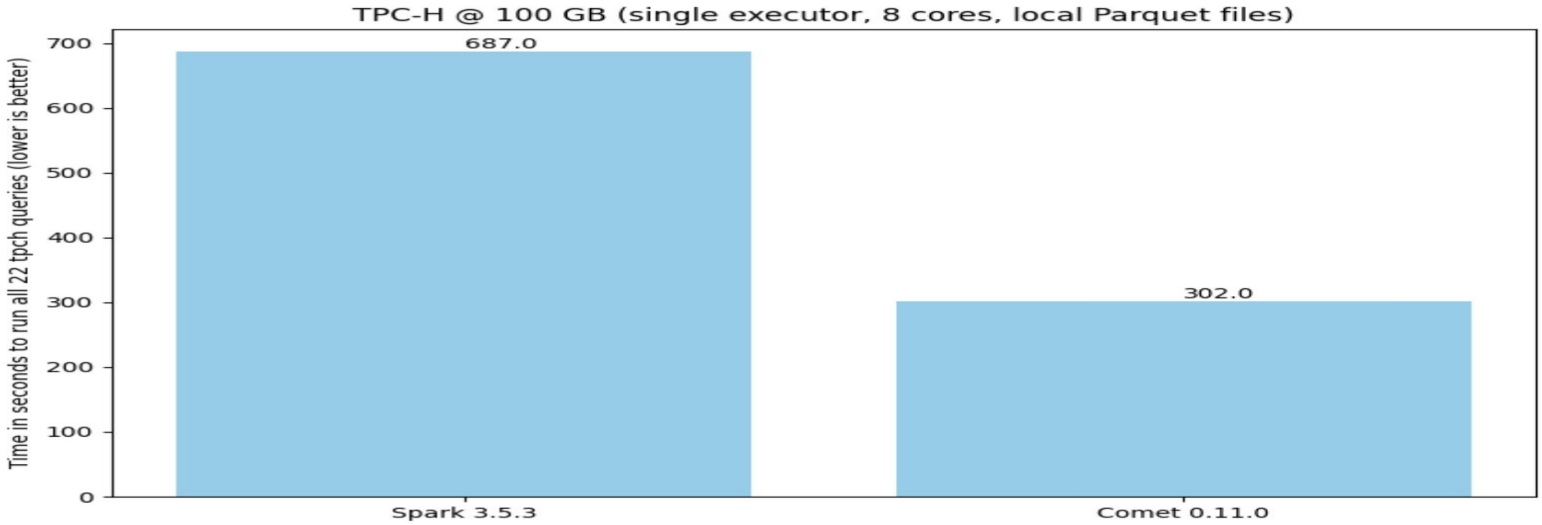
- Apache Spark is an open-source, distributed processing system used for big data workloads.
- Spark uses in-memory processing for its intermediate steps and also utilises caching to reduce physical disk I/O latency.
- It is **100 TIMES FASTER** than Hadoop MapReduce!

- The disadvantage of Spark is that it executes query operators as JVM Bytecode
- The query engine compiles the block of code to generate a Bytecode for each method
- Bytecode bundles all code application into a single .JAR file



DataFusion Comet

- Comet replaces JVM-Bytecode with native Rust.
- Native rust eliminates the overhead of Garbage Collector and also reduces the SIMD clock cycles.
- Comet also utilizes the Arrow columnar data format which is faster because of vector processing.



- Yet, as of today, not all the query operators run natively in Comet and instead fall back to Spark.
- Our job is to investigate which physical operators and expressions execute natively vs fall back to the JVM
- This is done by computing the coverage metrics over benchmark workloads (TPC-H, TPC-DS)

TPC-H Results

```
Q1      0.601 s
Q2      0.461 s
Q3      0.748 s
Q4      0.405 s
Q5      1.002 s
Q6      0.162 s
Q7      2.084 s
Q8      0.765 s
Q9      1.110 s
Q10     0.596 s
Q11     0.377 s
Q12     0.438 s
Q13     0.694 s
Q14     0.284 s
Q15     0.428 s
26/02/25 10:08:21 WARN CometExecRule: Comet cannot execute some parts of this plan natively (set spark.comet.explainFallback.enabled=false to disable this logging):
CometSort
+- CometSinkPlaceHolder
   +- CometExchange
      +- CometHashAggregate
         +- CometSinkPlaceHolder
            +- CometExchange
               +- CometHashAggregate
                  +- CometHashAggregate
                     +- CometSinkPlaceHolder
                        +- CometColumnarExchange
                           +- HashAggregate
                              +- Project
                                 +- BroadcastHashJoin
                                    :-  BroadcastHashJoin [COMET: BuildRight with LeftAnti is not supported]
                                    :  :- CometFilter
                                    :  :  +- CometScanWrapper
                                    :  +- BroadcastExchange
                                    :     +- CometProject
                                    :        +- CometFilter
                                    :           +- CometScanWrapper
                                    +- BroadcastExchange
                                       +- CometFilter
                                          +- CometScanWrapper

Comet accelerated 17 out of 23 eligible operators (73%). Final plan contains 0 transitions between Spark and Comet.
Q16     0.580 s
```

What’s up with the super high latency of Q21??

```
Q17     0.582 s
Q18     1.025 s
Q19     0.323 s
Q20     0.468 s
Q21    33.306 s
26/02/25 10:08:58 WARN CometExecRule: Comet cannot execute some parts of this plan natively (set spark.comet.explainFallback.enabled=false to disable this logging):
CometSort
+- CometSinkPlaceHolder
   +- CometColumnarExchange
      +- HashAggregate
         +- CometSinkPlaceHolder
            +- CometColumnarExchange
               +- HashAggregate
                  +- Project
                     +-  BroadcastHashJoin [COMET: BuildRight with LeftAnti is not supported]
                        :- CometFilter
                        :  :  +- Subquery
                        :  :     +- CometColumnarToRow
                        :  :        +- CometHashAggregate
                        :  :           +- CometExchange
                        :  :              +- CometHashAggregate
                        :  :                 +- CometProject
                        :  :                    +- CometFilter
                        :  :                       +- CometScan [native_iceberg_compat] parquet spark_catalog.default.customer_parquet
                        :  +- CometScanWrapper
                        +- BroadcastExchange
                           +- CometScanWrapper

Comet accelerated 14 out of 20 eligible operators (70%). Final plan contains 1 transitions between Spark and Comet.
Q22     0.875 s
```

The Left Semi Join operator with inequality is the issue

```
scala> val q_eq = """
     | select l1.l_orderkey, l1.l_suppkey
     | from lineitem l1
     | where exists (
     |   select 1 from lineitem l2
     |   where l2.l_orderkey = l1.l_orderkey
     |     and l2.l_suppkey = l1.l_suppkey
     | )
     | """
q_eq: String =
"
select l1.l_orderkey, l1.l_suppkey
from lineitem l1
where exists (
  select 1 from lineitem l2
  where l2.l_orderkey = l1.l_orderkey
    and l2.l_suppkey = l1.l_suppkey
)
"

scala> spark.sql(q_eq).count   // warm-up
res13: Long = 6001215

scala> val t1 = System.nanoTime
t1: Long = 624352591253816

scala> spark.sql(q_eq).count
res14: Long = 6001215

scala> println("NE secs = " + (System.nanoTime - t1)/1e9)
NE secs = 2.254417736
```

```
scala> val q_ne = """
     | select l1.l_orderkey, l1.l_suppkey
     | from lineitem l1
     | where exists (
     |   select 1 from lineitem l2
     |   where l2.l_orderkey = l1.l_orderkey
     |     and l2.l_suppkey <> l1.l_suppkey
     | )
     | """
q_ne: String =
"
select l1.l_orderkey, l1.l_suppkey
from lineitem l1
where exists (
  select 1 from lineitem l2
  where l2.l_orderkey = l1.l_orderkey
    and l2.l_suppkey <> l1.l_suppkey
)
"

scala> spark.sql(q_ne).count   // warm-up
res23: Long = 5786993

scala> val t1 = System.nanoTime
t1: Long = 624706606372793

scala> spark.sql(q_ne).count
res24: Long = 5786993

scala> println("NE secs = " + (System.nanoTime - t1)/1e9)
NE secs = 29.516249169
```

TPC-DS Results

- TPC-DS is a much larger benchmark and it consists of 99 Queries in total, which is 4.5 times more than the queries in the TPC-H.
- Nearly 40 queries fall back to JVM out of the total of 99
- The most common Fallback reasons are:

  (1) Unsupported Join types

  (2) Unsupported Aggregation Modes

  (3) Unsupported Window Functions

Next Steps

- Identify overhead bottlenecks in TPC-DS w.r.t Joins, Aggregates and Window Functions
- Propose concrete optimization targets for both the TPC-H & TPC-DS
- Implement performance improvements and feature extensions
- Finally benchmark the before vs after performance of operator performance w.r.t time taken and the native coverage.

Quiz Question

Why/How does DataFusion Comet accelerate Spark SQL?

(A): By executing query operators natively in JVM

(B): By executing query operators natively in Rust outside the JVM

(C): By rewriting all SQL queries as stored procedures

(D): By caching all the data in JVM heap memory

(E) By compressing data before sending it to the JVM

Image & Content References

slide-3
https://www.oracle.com/database/what-is-database/
https://www.geeksforgeeks.org/sql/what-is-sql/
https://aws.amazon.com/what-is/sql/
slide-4
https://datamozart.substack.com/p/parquet-file-format-everything-you
slide-5
https://aws.amazon.com/what-is/apache-spark/
slide-6
https://www.edureka.co/blog/java-bytecode/
slide-7
https://datafusion.apache.org/comet/